

**FRAME WARP: CLIENT-SIDE FRAME
REPROJECTION
FOR MOTION-TO-PHOTON LATENCY MITIGATION
IN BROWSER AND CLOUD-STREAMED RENDERING**

A PROJECT REPORT

submitted by

ISAAC

(Reg. No. URK23AI1035)

in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES

(Deemed to be University)

Coimbatore – 641114, Tamil Nadu, India

JUNE 2026

BONAFIDE CERTIFICATE

Certified that this project report titled “*Frame Warp: Client-Side Frame Reprojection for Motion-to-Photon Latency Mitigation in Browser and Cloud-Streamed Rendering*” is the bonafide work of **ISAAC (Reg. No. URK23AI1035)**, who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE FACULTY GUIDE

Name: _____

Designation: _____

SIGNATURE OF THE HEAD OF THE DEPARTMENT

Name: _____

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I express my sincere gratitude to my faculty guide, _____, for the guidance and encouragement extended throughout the course of this project.

I thank the Head of the Department and the faculty of the Department of Artificial Intelligence and Data Science, Karunya Institute of Technology and Sciences, for providing the environment and resources that made this work possible.

I also thank my family and friends for their constant support.

ISAAC

ABSTRACT

Motion-to-photon latency — the delay between physical input and the photons that reflect it — is the dominant feel-defining property of interactive rendering, and it worsens in exactly the situations modern systems create: heavy games running at low source frame rates, and cloud game streaming, where the rendered frame must additionally survive encoding, a network round trip, and decoding. This project demonstrates, measures, and analyses a client-side mitigation: frame reprojection ("frame warp"), in which the most recent rendered frame is warped every display refresh by the camera rotation that has occurred since the frame was drawn — the same family of technique as VR asynchronous timewarp and NVIDIA Reflex 2 Frame Warp.

Three artefacts are delivered. First, a browser-based shooting-range demo (Three.js/WebGL) in which a 30 FPS, deliberately delayed pipeline is reprojected to display rate, with honest orientation-based hit detection, live measured latency, and CSV export: the displayed view direction improves from approximately 91 ms stale to one display interval (~ 17 ms at 60 Hz). Second, the same compositor is split across a real network: a server window streams the rendered scene as WebRTC video while the player window reprojects the decoded stream using local input, with frame-exact pose synchronisation achieved by encoding each frame's identifier into its own pixels inside the guard-band margin. Measured end-to-end, the unwarped view-direction latency rises linearly with injected one-way delay (150/264/381 ms at 40/100/160 ms), while the warped view direction stays flat at ~ 17 ms, and is shown to be independent of delay jitter. Third, a deterministic replay instrument re-runs recorded or synthetic input traces through the full pipeline logic headlessly and bit-reproducibly; it validates an analytic error model (guard-band exhaustion bounds, reprojection linearisation error) and evaluates a velocity-adaptive guard band that reduces the rendering overhead of the warp margin from +73% to +9% on calm input while eliminating margin exhaustion on aggressive input.

The technique's limits are stated as precisely as its gains: rotation-only reprojection cannot compensate object motion, translation parallax, or interaction latency, and the warped-latency figure is a proxy floor that excludes hardware components no browser

can observe. These limitations define the future-work programme: hardware ground-truth instrumentation, a controlled user study, and depth-aware reprojection.

Keywords: *motion-to-photon latency, frame reprojection, asynchronous timewarp, cloud gaming, WebRTC, guard band, latency compensation*

TABLE OF CONTENTS

1. INTRODUCTION	8
1.1 The problem	8
1.2 The idea.....	8
1.3 Objectives	8
1.4 Scope and honesty principles.....	9
1.5 Organisation of the report.....	9
2. LITERATURE REVIEW	10
2.1 Image-based rendering and post-render warping.....	10
2.2 VR reprojection.....	10
2.3 Cloud gaming: systems and latency measurement	10
2.4 Latency compensation for cloud gaming.....	10
2.5 Frame generation and industry frame warp	11
2.6 Transport and contemporary work.....	11
3. SYSTEM DESIGN	12
3.1 The local demo: two clocks, one warp.....	12
3.2 The guard band	12
3.3 Honest hit detection	12
3.4 The cloud pipeline.....	12
3.5 Frame-exact pose synchronisation.....	13
3.6 End-to-end measurement in one clock.....	13
3.7 The replay instrument	13
4. THEORETICAL ANALYSIS	14
4.1 Latency decomposition.....	14
4.2 Jitter immunity	14
4.3 Error bound I: guard-band exhaustion.....	14
4.4 Error bound II: linearisation.....	15
4.5 Error bound III: translation parallax	15
5. IMPLEMENTATION	16
5.1 Stack.....	16
5.2 Module map	16
5.3 Verification	16
6. RESULTS AND DISCUSSION	18
6.1 Local demo.....	18
6.2 Cloud pipeline.....	18

6.3 Guard-band exhaustion: prediction vs measurement.....	19
6.4 The adaptive guard band.....	20
6.5 Measurement caveats.....	21
7. LIMITATIONS AND FUTURE WORK	22
7.1 What does not survive the network.....	22
7.2 Stated limitations	22
7.3 Future work.....	22
8. CONCLUSION.....	24
REFERENCES	25

(In Microsoft Word, right-click the table and choose “Update Field” to populate page numbers.)

1. INTRODUCTION

1.1 The problem

A display refreshes at a fixed rate, but the underlying 3D scene may be redrawn far less often — a demanding game at 30 FPS produces a new frame only every 33 ms — and every stage of a real pipeline (input sampling, simulation, rendering, queueing, and in cloud streaming: encoding, transmission, decoding) holds the user's input a little longer before it becomes visible. The accumulated delay is called motion-to-photon (MtP) latency. For camera control it is directly felt: the view lags the hand. Prior measurement studies place cloud gaming latencies in the 100–300 ms range [7][8], with measurable degradation of player performance and quality of experience well below that [12][13][14].

1.2 The idea

Mouse input is available at hundreds of hertz even when frames are not. Frame reprojection runs two clocks: the scene is rendered into a texture at its slow native rate, and at every display refresh the latest texture is warped — shifted by the camera rotation that has occurred since the frame was rendered — so the displayed view direction tracks the freshest input even though the pixels are old. The technique descends from post-rendering 3D warping [4] and image-based rendering [5][6], reached consumers as VR asynchronous timewarp [1][2][3], and is the substance of NVIDIA's announced Reflex 2 Frame Warp [23].

1.3 Objectives

- Build a browser demo that makes the latency problem and the warp's effect directly visible and falsifiable (a tracking-and-shooting task with honest, orientation-based hit detection).
- Split the same pipeline across a real network — server renders and streams video; client reprojects the decoded stream with local input — with no server modification beyond rendering a wider field of view, and measure end-to-end latency under controlled delay and jitter.
- Formalise what rotation-only reprojection removes and where it breaks (guard-band exhaustion, linearisation error, parallax), and validate the analysis with a deterministic replay instrument.

- Evaluate one extension — a velocity-adaptive guard band — against its fixed baseline using replay benchmarks.

1.4 Scope and honesty principles

Two principles run through the project. First, measure rather than assert: every latency number is computed from real timestamps, exported as CSV, and (for the analytic claims) reproduced by automated tests. Second, state limits as precisely as gains: the warp compensates camera rotation only; it cannot reconstruct occluded geometry, compensate object motion over a video stream, or shorten the latency of discrete interactions. Where a measured quantity is a proxy rather than ground truth (the warped-view latency excludes mouse polling, GPU queueing and scanout, which a browser cannot observe), the report says so explicitly.

1.5 Organisation of the report

Chapter 2 reviews related work. Chapter 3 presents the system design of the local demo, the cloud-streaming pipeline, and the replay instrument. Chapter 4 develops the theoretical analysis. Chapter 5 details the implementation. Chapter 6 presents results. Chapter 7 discusses limitations and future work, and Chapter 8 concludes.

2. LITERATURE REVIEW

2.1 Image-based rendering and post-render warping

View interpolation [5] and plenoptic warping [6] established that nearby views can be synthesised from rendered images instead of re-rendering; post-rendering 3D warping [4] applied this directly to latency, warping a finished frame to a newer viewpoint. All of this line assumes local access to renderer outputs, often including depth.

2.2 VR reprojection

Carmack's latency-mitigation analysis [1] brought timewarp to consumer VR; van Waveren [2] made it asynchronous and scheduled on consumer GPUs; asynchronous spacewarp [3] added extrapolation for object motion. The guard-band render margin and the rotation-only honesty caveats used in this project are inherited from this literature. FlashBack [26] and Furion [27] brought related techniques to mobile VR via precomputation and split rendering. In all cases the compositor owns the GPU, the engine, and undistorted frames — none of which a browser client of a video stream possesses.

2.3 Cloud gaming: systems and latency measurement

Chen et al. [7] established the methodology for measuring cloud gaming latency on commercial systems; Choy et al. [8] showed proximity alone cannot solve it; Jarschel et al. [9] and Claypool and Finkel [13] quantified the QoE cost; Claypool and Claypool [12] and Beigbender et al. [14] established per-genre latency sensitivity, with first-person camera control the hardest case. Shea et al. [10] surveyed the architecture; GamingAnywhere [11] provided the open research platform; Kamarainen et al. [15] measured what end-to-end budget is required for imperceptible latency; Carrascosa and Bellalta [16] characterised a production service. These works measure and decompose latency; they do not remove terms of it.

2.4 Latency compensation for cloud gaming

The closest prior art is server-side or engine-side. Outatime [17] speculatively renders probable futures on the server and ships them ahead of input — requiring engine integration and server compute per speculation branch. Kahawai [18] splits rendering between server and a client GPU running the same engine. Sabet et al. [19] adapt game mechanics to latency rather than reducing it; lag-compensation techniques for FPS

games [20] adjust hit registration rather than the displayed view. None of these reproject the video stream itself, on the client, without engine access — the gap this project occupies.

2.5 Frame generation and industry frame warp

DLSS 3 [21] and FSR 3 [22] generate intermediate frames locally using engine motion vectors; Reflex 2 Frame Warp [23] is the industrial statement of this project's thesis — warp the newest frame to the freshest input — but is local-GPU, proprietary, and unshipped in games as of June 2026. No streaming variant has been published.

2.6 Transport and contemporary work

WebRTC [24] is currently the only browser path to sub-second interactive video; WebTransport-based game streaming systems are emerging [25]. Most recently, Siekkinen and Kamarainen [28] stream rendered scenes as 3D Gaussians so the client can re-render views — explicitly motivated by the limits of image-space reprojection of 2D video, which their work treats as the baseline category. That approach abandons commodity video codecs; this project stays inside them, and the two are therefore complementary: a mature Gaussian-streaming system must beat exactly the commodity-video baseline this project measures. Whether rotation-compensated reprojection of commodity video streams can achieve perceptually imperceptible latency [15] — without the bandwidth and server-modification requirements of representation-level approaches — remains an open empirical question, and is the motivating question of this work's research programme.

3. SYSTEM DESIGN

3.1 The local demo: two clocks, one warp

The local demo is a single-screen shooting range. The fast clock samples pointer-lock mouse input on every event (125–1000 Hz) and maintains the camera yaw/pitch. The slow clock renders the 3D scene at a capped 30 FPS into an off-screen texture, deliberately using an orientation from a configurable interval in the past (default 80 ms; the always-on simulated condition of a heavy pipeline). Every display refresh, a fullscreen-quad compositor draws the newest texture through the warp shader, shifted by the angular delta between the freshest input and the orientation the frame was rendered with: $\Delta u = -\Delta \text{yaw} / \text{fov}_x$, $\Delta v = \Delta \text{pitch} / \text{fov}_y$. Pressing W toggles only this shift; everything else is identical between the two modes.

3.2 The guard band

A shifted image needs pixels beyond its own edge. The scene is therefore rendered at a wider field of view than displayed, and the shader shows the central crop: $\text{sampleUV} = g + (uv + \Delta) \cdot (1 - 2g)$, with margin $g = 0.12$ per side. The render FOV uses the tangent-exact relation $\tan(F_{\text{render}}/2) = \tan(F_{\text{display}}/2)/(1 - 2g)$, so the displayed crop equals the intended display FOV exactly. Edge clamping remains only as the fallback when motion exceeds the margin — the failure mode analysed in Chapter 4.

3.3 Honest hit detection

The shooter exists to make latency falsifiable, so the hit test must not flatter the warp. A click fires one ray, from the camera orientation the screen is currently displaying: the current orientation when warp is on, the lagged orientation when off — against identical targets whose positions advance only on the render tick (so displayed and tested positions always agree). The outcome can differ between modes only through the view-direction latency the warp removes, which is exactly the quantity under test.

3.4 The cloud pipeline

The same compositor, split across a network. A server window renders the range at 30 FPS into a fixed 1280×720 canvas and streams it as WebRTC video; a player window receives the stream into a video element, uses it as the warp texture, and forwards its own pointer-lock input to the server as full-pose, latest-wins packets (~120 Hz) over an unreliable, unordered data channel — the configuration a real game-streaming control

channel uses. The server applies a configurable one-way delay (with optional sinusoidal jitter, 40–140 ms, monotonicity-preserving) to both the outgoing frames and the incoming input, so the round trip is realistic in both directions.

3.5 Frame-exact pose synchronisation

To warp a decoded frame correctly, the client must know the camera pose that frame was rendered with — but video frames and data-channel packets arrive by independent paths, and browser frame metadata does not identify frames exactly (this Chromium exposes `receiveTime` but not `captureTime`). The design solves this in the pixels themselves: the server bakes a 16-bit frame identifier into a 4×4 grid of black/white cells in the frame's corner — inside the guard-band margin, so the displayed crop hides it — and the client reads the identifier back from the decoded video and looks up the pose that travelled on the data channel. The match is exact by construction. Timestamp-based matching ("Plan A") is retained and scored live against the pixel tag: it agrees ~99.5% on a calm network, degrading to ~95% under jitter — the measured justification for making the tag primary.

3.6 End-to-end measurement in one clock

Cross-machine clock comparison is avoided entirely. The client timestamps each forwarded input packet by sequence number; the server echoes, in each frame's pose packet, which input sequence the frame's camera used; when the displayed frame's pixel tag resolves that pose, the client computes `now - sentAt(seq)` — the full uplink, render-wait, encode, delay, decode loop, measured entirely in the client's clock.

3.7 The replay instrument

Every experiment beyond the live demos runs on a deterministic replay system. An input trace — a timestamped pose stream, recorded from the live demo (T key) or generated synthetically — is replayed through a headless simulator that drives the actual LagSim class through the same display-tick/render-tick structure as the live loop, models the warp delta and its guard-band clamp, and emits per-tick CSV. The simulator is bit-reproducible across runs and processes, so any configuration comparison is perfectly controlled: same input, different pipeline. All figures in Chapter 6 are generated from this instrument or from the live demos' CSV exports.

4. THEORETICAL ANALYSIS

4.1 Latency decomposition

End-to-end motion-to-photon latency for a streamed or heavily pipelined renderer decomposes as $L_{\text{total}} = L_{\text{input}} + L_{\text{uplink}} + L_{\text{render}} + L_{\text{encode}} + L_{\text{downlink}} + L_{\text{decode}} + L_{\text{composite}}$. For camera rotation, client-side reprojection replaces every term except the last: the compositor reads the freshest local pose every refresh and shifts the newest decoded frame by the pose delta, so the displayed view direction reflects input at most one display interval old, regardless of how old the pixels are. Every other interaction — shooting, object motion, translation — retains the full sum; nothing client-side can shortcut a state change the client cannot compute.

4.2 Jitter immunity

Let the displayed frame at composite time t have arbitrary age $A(t)$. If the required delta is within the guard margin, the displayed view direction equals the input pose sampled at t exactly: the compositor computes $\Delta = \text{pose_now} - \text{pose_frame}$, where pose_frame is recovered exactly via the pixel tag, and applies it in full. The frame's age determines only which pose enters the subtraction — the subtraction then cancels it. Perceived rotation latency is therefore independent of both the mean and the variance of frame age. Two honest conditions apply: jitter does widen the distribution of Δ and thus raises the probability of margin exhaustion, and pose recovery must be exact (with timestamp matching instead of the tag, jitter degrades match accuracy, as measured in §3.5).

4.3 Error bound I: guard-band exhaustion

The largest compensable rotation is $\Delta_{\text{max}} = (g/(1-2g)) \cdot F = 16.98^\circ$ at $g = 0.12$ and the demo's 107.5° horizontal FOV. A displayed frame's age in steady state lies in $[L, L + T_{\text{render}} + T_{\text{display}}]$: it is L old when rendered, ages up to one render interval before replacement, and the replacement can land one display tick late because renders are quantised to display ticks. At constant angular velocity ω , clamping therefore begins somewhere in $[\Delta_{\text{max}}/(L+T_r+T_d), \Delta_{\text{max}}/L]$ — for the default configuration, between 130.6 and 212.2 deg/s. The simulator measures onset at ≈ 143 deg/s, inside the predicted band (Figure 2). Notably, the display-tick term in the age bound was found by the instrument: the first draft of the bound omitted it, and a failing automated test exposed the omission.

4.4 Error bound II: linearisation

The shader's shift is uniform in screen UV — linear in angle — while a perspective image is linear in the tangent of angle. To first order, the per-radian error of the uniform shift at angle θ from screen centre is $e(\theta)/\delta = \sec^2\theta/(2 \cdot \tan(F/2)) - 1/F$: exactly zero at $\theta \approx \pm 34^\circ$ for the demo FOV, over-sliding by ≈ 213 px/rad (at 1280 px width) at centre and under-sliding by roughly $3\times$ that magnitude, opposite sign, at the edge. Since the applied delta is the full frame-age delta ($\delta \approx \omega \cdot A$), brisk tracking at 60 deg/s with a 100 ms-old frame misplaces centre pixels by ≈ 22 px — converging to zero as rotation stops, i.e. precisely when aim must be precise. The exact correction is a per-pixel tangent remap (a one-dimensional homography of the same cost class); it is left as future work because all measured results used the linear form.

4.5 Error bound III: translation parallax

Rotation-only warp models the view as a pure rotation about the optical centre. Camera translation v over frame age A shifts a point at depth d by parallax angle $\approx v_{\perp} \cdot A/d$, none of which the warp applies; the screen error is $\approx (W/F) \cdot v_{\perp} \cdot A/d$ pixels. Walking speed against 2 m geometry at $A = 150$ ms gives errors near 77 px; the same motion against 50 m geometry, ≈ 3 px. This term is stated analytically only — the demo camera never translates — and is the quantitative motivation for depth-aware reprojection as future work, where streamed depth turns the term from error into signal.

5. IMPLEMENTATION

5.1 Stack

Three.js (r169, vendored locally — the demo runs fully offline) over WebGL, vanilla ES modules with no build step, served by a small no-cache static server. The cloud pipeline uses browser WebRTC with localStorage-based same-origin signalling (offer/answer exchanged through storage events; non-trickle ICE; session identifiers make the handshake self-healing across reloads). All measurement code uses `performance.now()` timestamps and exports CSV.

5.2 Module map

Local demo (src/): `input.js` (fast clock), `lag.js` (slow clock + injected delay), `scene.js` / `targets.js` / `raycast.js` (the range, moving targets, honest hit rays), `warp-target.js` / `warp-shader.js` / `quad-render.js` (render-to-texture and the reprojection shader with guard band and motion-vector support), `latency.js` / `chart.js` / `recorder.js` (measurement, live chart, CSV), `hud.js`, `controls.js`, `main.js` (the conductor).

Cloud (src/cloud/): `signaling.js`, `frame-tag.js` (the 16-bit pixel tag codec — pure functions, unit-tested), `pose-sync.js` (pose ring buffer; exact byFrameId primary, timestamp variants scored against it), `server-main.js` (capture pipeline, delay queue, jitter, input delay), `client-main.js` (video → warp compositor, input forwarding, e2e measurement), `cloud-recorder.js` (CSV with per-mode summary statistics).

Replay instrument (src/replay/, bench/): `trace.js` (trace format, browser recorder, deterministic synthetic generators), `pipeline-sim.js` (the headless simulator; drives the real LagSim), `adaptive-guard.js` (the Option C policy), `bench/run.js` and `bench/adaptive.js` (command-line benchmarks emitting CSV).

5.3 Verification

Thirty-nine unit tests (vanilla Node, no framework) cover the pure mathematics: the tangent-exact guard-band FOV, the shader's UV transform, the lag buffer, ray-based hit testing, the frame-tag codec, pose synchronisation including clock-offset estimation, recorder percentiles, trace semantics, simulator determinism, the clamp-onset bound against the simulator, the linearisation zero-crossing, and the adaptive-guard policy. The warp's pixel-exactness was verified directly in the browser: a 0.1 rad camera step

produces a warped canvas equal to the unwarped canvas shifted by the predicted 68 px, with the residual below the temporal noise floor.

6. RESULTS AND DISCUSSION

6.1 Local demo

At the default condition (30 FPS source, 80 ms injected pipeline delay, 60 Hz display), the measured view-direction staleness without warp is ≈ 91 ms; with warp the displayed view direction is one display interval old, ≈ 17 ms. The tracking-and-shooting task makes the difference behaviourally visible: while tracking a moving target, shots fired without warp land where the aim was ~ 91 ms ago and miss; with warp they hit. The replay instrument reproduces the same relationship across the full delay range (Figure 1): the without-warp series rises linearly with injected delay (slope 1, offset bounded by the render interval plus a display tick), while the warp series is flat by construction.

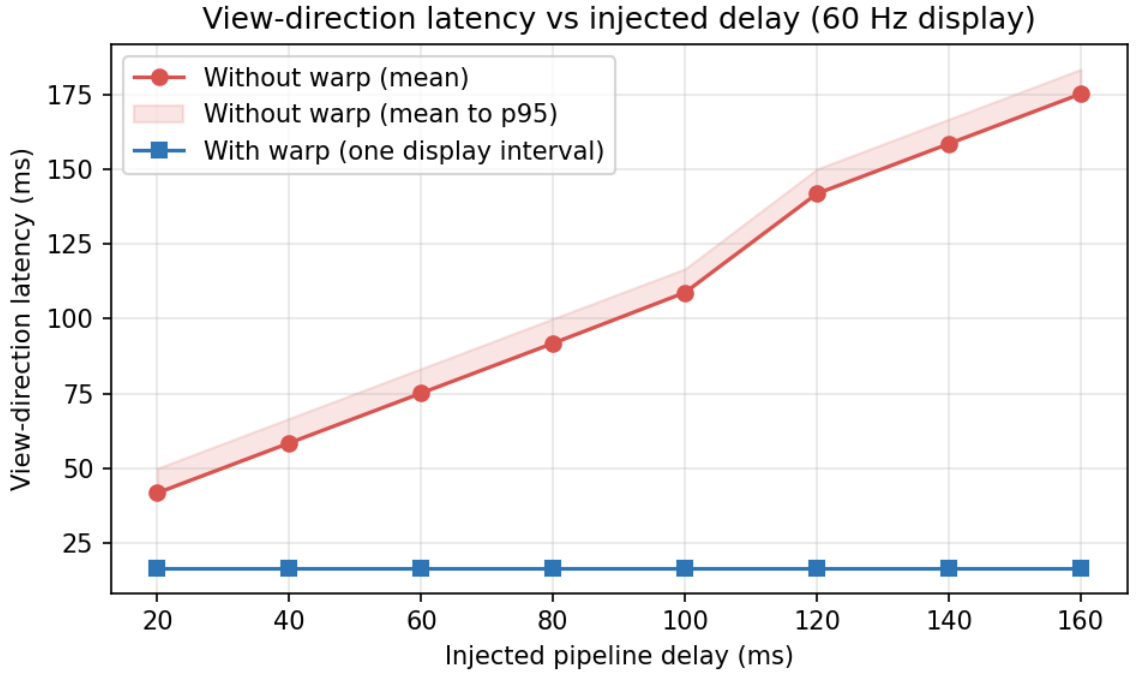


Figure 1. View-direction latency vs injected pipeline delay (replay instrument, wander trace, 60 Hz display). The without-warp series rises with delay; the warped view direction stays at one display interval.

6.2 Cloud pipeline

Table 1 shows the measured end-to-end behaviour on loopback with controlled one-way delay. Without warp, end-to-end view-direction latency tracks the round trip linearly (slope ≈ 2 plus ≈ 65 ms of codec and frame-interval overhead). With warp, the perceived view-direction latency is flat at one display interval regardless of network delay — and, per §4.2, regardless of jitter: with jitter enabled the unwrapped stream is visibly unstable while the warped view is indistinguishable from a calm network.

One-way delay (ms)	E2E latency, warp off (ms)	View latency, warp on (ms)
40	~150	~17
100	~264	~17
160	~381	~17

Table 1. Measured cloud-pipeline latency on loopback (CSV export, per-mode means).

Pose synchronisation: the pixel frame tag resolved the displayed frame's pose exactly throughout; the timestamp-matching alternative agreed with the tag on $\sim 99.5\%$ of frames on a calm network and $\sim 95\%$ under jitter, which is the measured justification for the tag-primary design.

6.3 Guard-band exhaustion: prediction vs measurement

Figure 2 sweeps constant angular velocity through the simulator. Clamping begins at ≈ 143 deg/s, inside the analytic bounds of §4.3 (shaded); above onset, the clamp rate saturates and the residual view-direction error grows linearly with the velocity excess. This is the project's theory-meets-instrument result: the error model predicts the measured artifact onset within the stated bounds, and the comparison is an automated test.

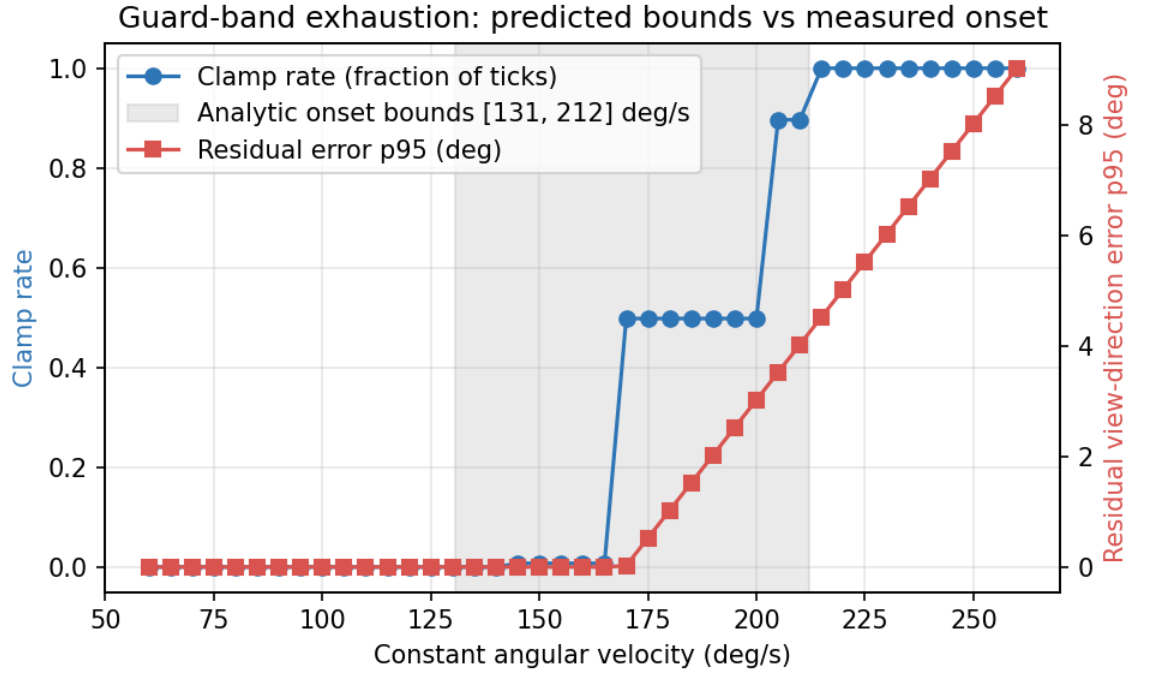


Figure 2. Guard-band exhaustion under constant angular velocity: measured clamp rate and residual error, with the analytic onset bounds shaded.

6.4 The adaptive guard band

The fixed 12% margin costs +73% rendered pixels at all times and is still exhausted by aggressive motion. The adaptive policy sizes the margin per rendered frame from the worst angular speed observed in the preceding 500 ms (safety factor 1.2, bounds [0.02, 0.20]), inverting the margin equation of §4.3. Figure 3 and Table 2 summarise the result: on calm input the pixel overhead falls to +33% $\rightarrow$ +9% (depending on trace) with zero clamping; on hot input the policy grows past the fixed margin during bursts and eliminates the clamping the fixed margin suffers (8.6% of ticks, residual errors up to 17.6°), paying the pixel cost only while the burst lasts.

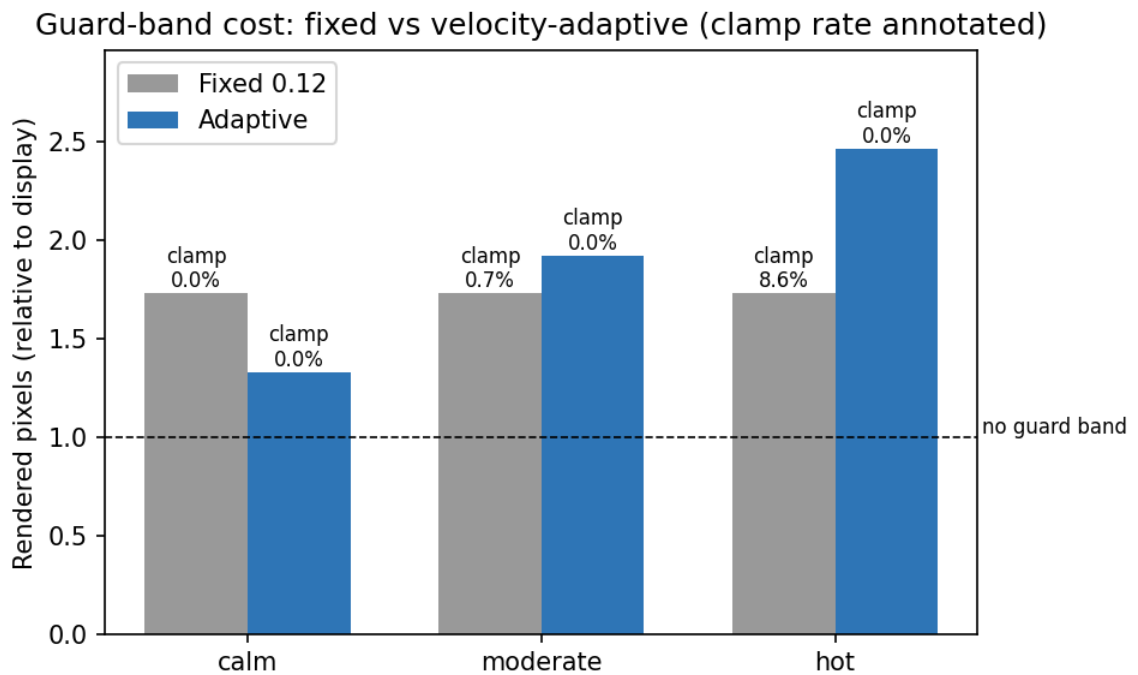


Figure 3. Rendered-pixel cost of the guard band, fixed vs velocity-adaptive, with clamp rates annotated (replay benchmark, three wander traces).

Trace	Policy	Pixel cost (×display)	Clamp rate
calm	fixed 0.12	1.73	0%
calm	adaptive	1.33	0%
moderate	fixed 0.12	1.73	0.7%
moderate	adaptive	1.92	0%
hot	fixed 0.12	1.73	8.6%
hot	adaptive	2.47	0%

Table 2. Fixed vs adaptive guard band (replay benchmark). The adaptive policy wins on cost or on quality on every trace, and on both for calm input.

6.5 Measurement caveats

The two latency series are not symmetric measurements. The without-warp figure is a directly measured staleness: $\text{now} - (\text{timestamp of the input the displayed frame reflects})$. The with-warp figure is the display-frame interval — a proxy floor expressing that the warp re-applies the freshest input every refresh. It excludes mouse polling, GPU queueing and display scanout, none of which a browser can observe; establishing the absolute motion-to-photon figure requires external hardware (photodiode or high-speed camera), which is the first item of the future-work programme. All comparative claims in this report are between quantities measured the same way.

7. LIMITATIONS AND FUTURE WORK

7.1 What does not survive the network

Capability	Local demo	Cloud demo	Why
Rotation warp	Yes	Yes	Needs only the pose delta
Guard band	Yes	Yes	Server renders wide FOV; client crops
Motion vectors	Yes	No	YUV 4:2:0 video has no channel for per-pixel velocity; lossy coding would destroy its precision
De-ghosting clamp	Yes	Inert	With no velocity term the clamp is mathematically a no-op; cloud edge defence is the guard band alone

Table 3. Honest capability table.

7.2 Stated limitations

- Rotation only: object motion, disocclusion and translation parallax are not compensated (bounds in §4.4–4.5); the moving target visibly steps at the source rate in the cloud demo.
- The warped-latency figure is a proxy floor (§6.5), not hardware motion-to-photon.
- The adaptive guard band is a simulation-level result: a real video encoder reacts to mid-stream FOV changes (rate control, references), and the surviving saving must be measured in the live cloud pipeline before being quoted as a systems result.
- The window-max velocity estimator adapts one frame late to a cold flick from rest; the safety factor and margin floor bound, but do not eliminate, first-frame clamping on pathological input.

7.3 Future work

- Hardware ground truth: photodiode/Arduino or high-speed camera instrumentation to calibrate the software measurements (the prerequisite for any absolute latency claim).

- A controlled user study (within-subjects, warp $\times$ delay $\times$ jitter) measuring hit rate, time-to-acquire and tracking error — the component that would convert the engineering result into a perceptual one, answering whether commodity-video reprojection reaches imperceptible latency [15].
- Depth-aware streamed reprojection: a low-resolution depth buffer over the data channel turns the parallax error term of §4.5 into signal, at a measurable bandwidth/artifact trade-off.
- A tangent-exact per-pixel remap replacing the linear UV shift (§4.4), and porting the transport to WebTransport [25] as it matures.

8. CONCLUSION

This project built, measured and analysed a client-side answer to motion-to-photon latency that works at the only place a streaming client can act: the final composite. A browser demo makes the problem and the fix directly experienceable and falsifiable; the same compositor, split across WebRTC, shows that a video stream that is genuinely hundreds of milliseconds old can be displayed with a view direction one display interval old — flat against network delay and provably indifferent to jitter — using no server modification beyond a wider field of view and an in-band frame tag. The analysis states exactly where the technique breaks (margin exhaustion beyond ~ 143 deg/s at the default configuration; linearisation shear during fast rotation; parallax under translation), and a deterministic replay instrument turns those statements into automated tests and reproducible benchmarks, including a velocity-adaptive guard band that removes most of the technique's standing GPU cost.

The result is a working system and a defensible claim: for camera rotation in cloud-streamed rendering, the latency the network adds does not have to be the latency the player feels.

REFERENCES

- [1] J. Carmack, "Latency Mitigation Strategies," AltDevBlogADay, 2013.
- [2] J. M. P. van Waveren, "The Asynchronous Time Warp for Virtual Reality on Consumer Hardware," in Proc. 22nd ACM Symposium on Virtual Reality Software and Technology (VRST), 2016.
- [3] Oculus VR, "Asynchronous Spacewarp," developer blog, 2016.
- [4] W. R. Mark, L. McMillan, and G. Bishop, "Post-Rendering 3D Warping," in Proc. Symposium on Interactive 3D Graphics (I3D), 1997.
- [5] S. E. Chen and L. Williams, "View Interpolation for Image Synthesis," in Proc. SIGGRAPH, 1993.
- [6] L. McMillan and G. Bishop, "Plenoptic Modeling: An Image-Based Rendering System," in Proc. SIGGRAPH, 1995.
- [7] K.-T. Chen, Y.-C. Chang, P.-H. Tseng, C.-Y. Huang, and C.-L. Lei, "Measuring the Latency of Cloud Gaming Systems," in Proc. ACM Multimedia, 2011.
- [8] S. Choy, B. Wong, G. Simon, and C. Rosenberg, "The Brewing Storm in Cloud Gaming: A Measurement Study on Cloud to End-User Latency," in Proc. NetGames, 2012.
- [9] M. Jarschel, D. Schlosser, S. Scheuring, and T. Hossfeld, "An Evaluation of QoE in Cloud Gaming Based on Subjective Tests," in Proc. IMIS, 2011.
- [10] R. Shea, J. Liu, E. C.-H. Ngai, and Y. Cui, "Cloud Gaming: Architecture and Performance," IEEE Network, 2013.
- [11] C.-Y. Huang, C.-H. Hsu, Y.-C. Chang, and K.-T. Chen, "GamingAnywhere: An Open Cloud Gaming System," in Proc. ACM MMSys, 2013.
- [12] M. Claypool and K. Claypool, "Latency and Player Actions in Online Games," Communications of the ACM, 2006.
- [13] M. Claypool and D. Finkel, "The Effects of Latency on Player Performance in Cloud-Based Games," in Proc. NetGames, 2014.
- [14] T. Beigbeder, R. Coughlan, C. Lusher, J. Plunkett, E. Agu, and M. Claypool, "The Effects of Loss and Latency on User Performance in Unreal Tournament 2003," in Proc. NetGames, 2004.
- [15] T. Kamarainen, M. Siekkinen, A. Yla-Jaaski, W. Zhang, and P. Hui, "A Measurement Study on Achieving Imperceptible Latency in Mobile Cloud Gaming," in Proc. 8th ACM Multimedia Systems Conference (MMSys), 2017.
- [16] M. Carrascosa and B. Bellalta, "Cloud-Gaming: Analysis of Google Stadia Traffic," Computer Communications, 2022.
- [17] K. Lee, D. Chu, E. Cuervo, J. Kopf, Y. Degtyarev, S. Grizan, A. Wolman, and J. Flinn, "Outatime: Using Speculation to Enable Low-Latency Continuous Interaction for Mobile Cloud Gaming," in Proc. 13th ACM MobiSys, 2015.

- [18] E. Cuervo, A. Wolman, L. P. Cox, K. Lebeck, A. Razeen, S. Saroiu, and M. Musuvathi, "Kahawai: High-Quality Mobile Gaming Using GPU Offload," in Proc. ACM MobiSys, 2015.
- [19] S. Shafiee Sabet, S. Schmidt, S. Zadtootaghaj, B. Naderi, C. Griwodz, and S. Moller, "A Latency Compensation Technique Based on Game Characteristics to Mitigate the Influence of Delay on Cloud Gaming Quality of Experience," in Proc. 11th ACM MMSys, 2020.
- [20] Z. Li, H. Melvin, R. Bruzgiene, P. Pocta, L. Skorin-Kapov, and A. Zgank, "Lag Compensation for First-Person Shooter Games in Cloud Gaming," in Autonomous Control for a Reliable Internet of Services, LNCS 10768, Springer, 2018.
- [21] NVIDIA, "DLSS 3: AI-Powered Frame Generation," technical overview, 2022.
- [22] AMD, "FidelityFX Super Resolution 3 and Fluid Motion Frames," technical overview, 2023.
- [23] NVIDIA, "Reflex 2 with Frame Warp," announced CES 2025 (not shipped in games as of June 2026).
- [24] H. Alvestrand, "Overview: Real-Time Protocols for Browser-Based Applications," RFC 8825, 2021.
- [25] M. Nguyen et al., "A WebTransport-based System for Real-Time Game Streaming," in Proc. 6th ICCVCI, 2025.
- [26] K. Boos, D. Chu, and E. Cuervo, "FlashBack: Immersive Virtual Reality on Mobile Devices via Rendering Memoization," in Proc. ACM MobiSys, 2016.
- [27] Z. Lai, Y. C. Hu, Y. Cui, L. Sun, and N. Dai, "Furion: Engineering High-Quality Immersive Virtual Reality on Today's Mobile Devices," in Proc. ACM MobiCom, 2017.
- [28] M. Siekkinen and T. Kamarainen, "Streaming Real-Time Rendered Scenes as 3D Gaussians," arXiv:2604.02851, 2026.